🧠 VIRQA — AI Interview Platform

Complete System Architecture & Flow Documentation

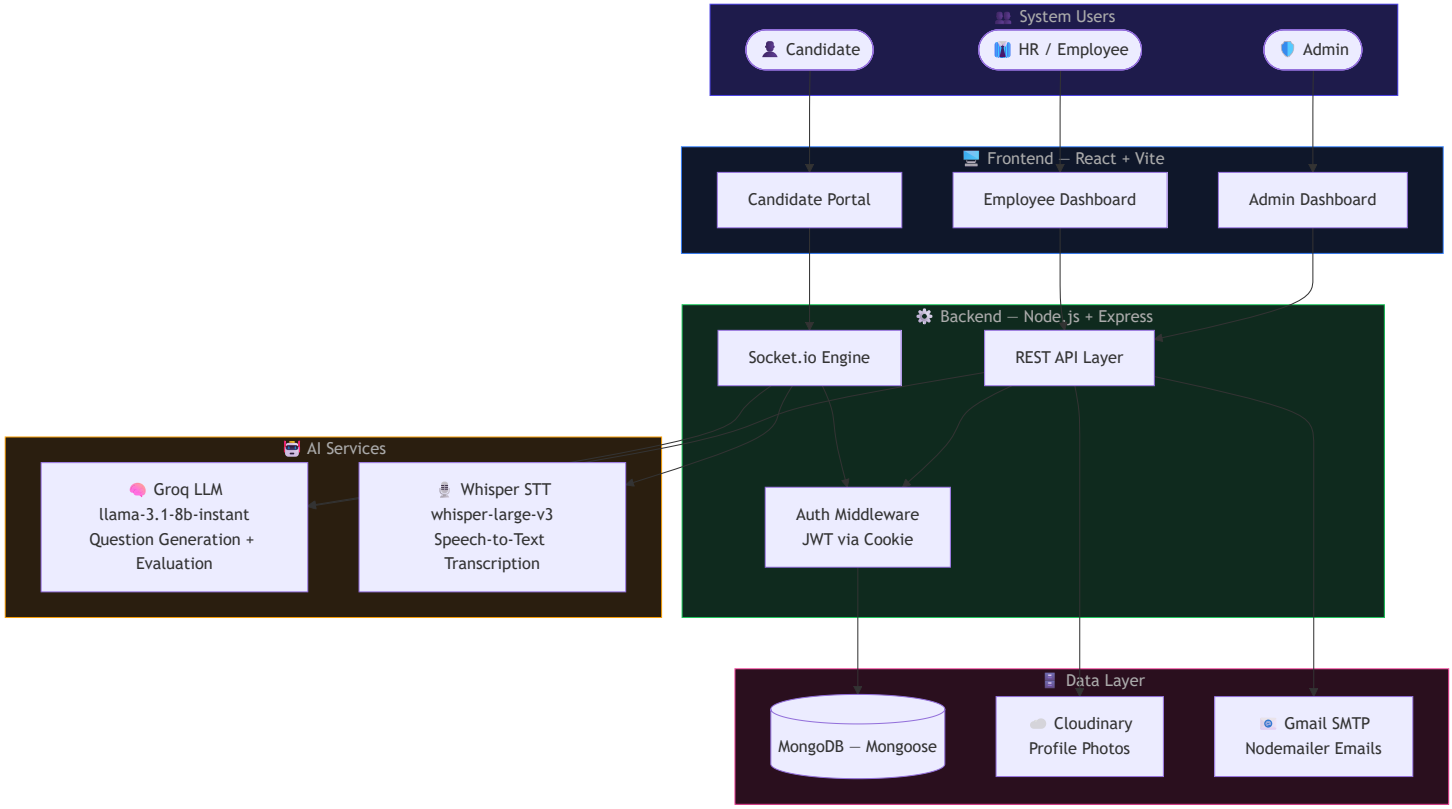
> **Version:** 1.0.0 | **Platform:** Full-Stack Web Application | **AI-Powered:** Yes
> **Institution:** University of Central Punjab (UCP) — Final Year Project (F22)
> **Author:** Muhammad Ummar | **Category:** Artificial Intelligence × Human Resources

This document provides an exhaustive technical reference for the VIRQA AI Interview Platform — covering every actor, data flow, API contract, AI service pipeline, database schema, and real-time Socket.io event across all 9 phases of the interview lifecycle.

📋 Table of Contents

0. System Overview

VIRQA is an AI-powered, end-to-end interview automation platform. It enables HR employees to schedule technical interviews, invites candidates automatically via email, and conducts fully automated AI interviews using voice recognition and LLM-based evaluation — all in real-time.



1. Technology Stack

Actors & Their Interfaces

Actor	Dashboard	Primary Actions
Admin	Admin Dashboard — React + Vite	Audit interviews, manage employees, view global reports
HR / Employee	Employee Dashboard — React + Vite	Create sessions, invite candidates, view results
Candidate	Candidate Portal — React + Vite	Accept invite, take AI interview, view their scores

Technology Reference

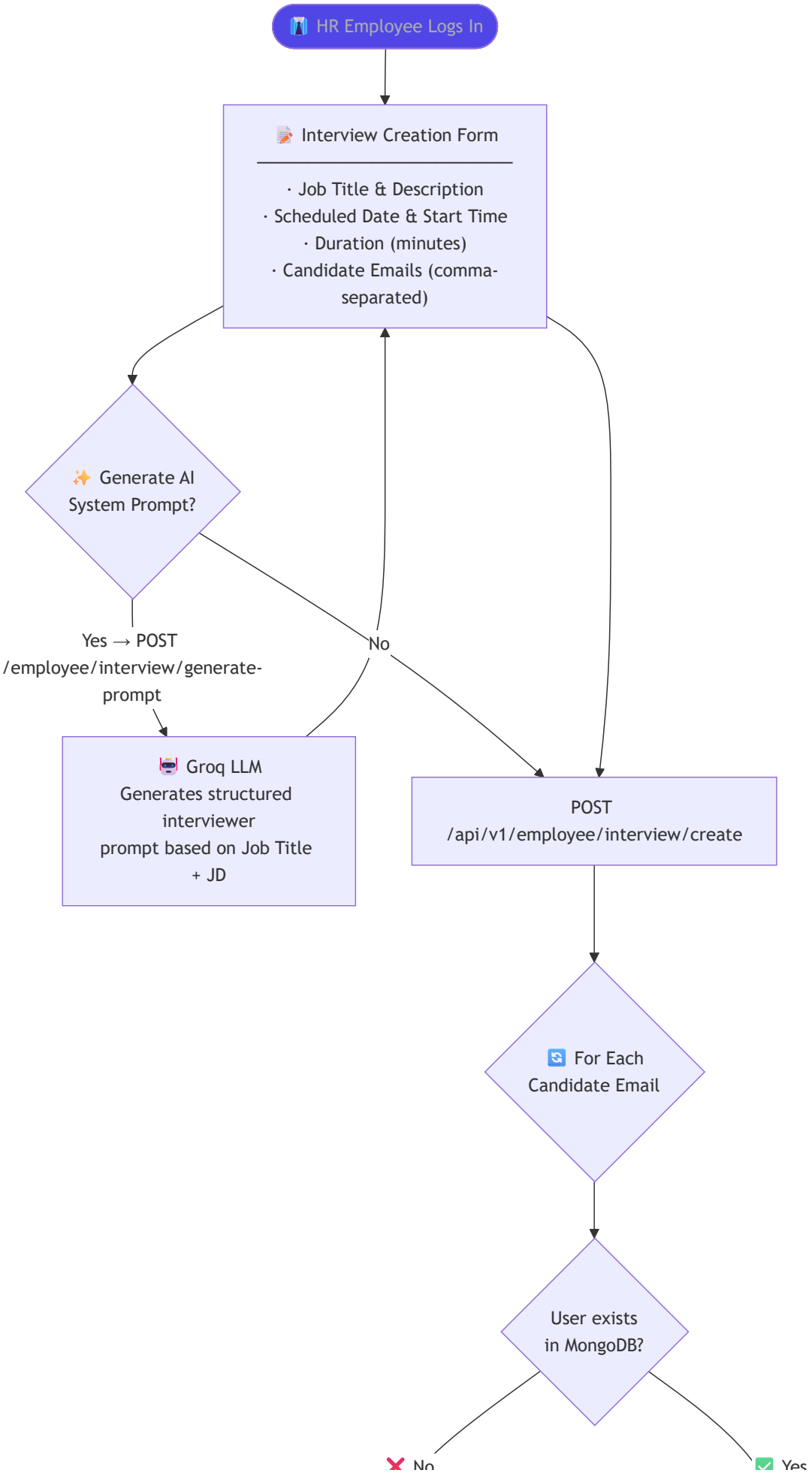
Layer	Technology	Purpose
Frontend	React 18 + Vite + TanStack Query	UI rendering, API state management
Real-time	Socket.io (Client + Server)	Bidirectional audio/event streaming
Backend	Node.js + Express 5	REST API + socket event processing
Auth	JWT (stored in HttpOnly Cookie)	Secure session management
LLM — Questions	Groq API · `llama-3.1-8b-instant`	Generates dynamic interview questions
LLM — Evaluation	Groq API · `llama-3.1-8b-instant`	Scores and provides feedback on answers
Speech-to-Text	Groq Whisper · `whisper-large-v3`	Converts candidate audio to text
Database	MongoDB + Mongoose (Discriminator pattern)	Document storage with inheritance

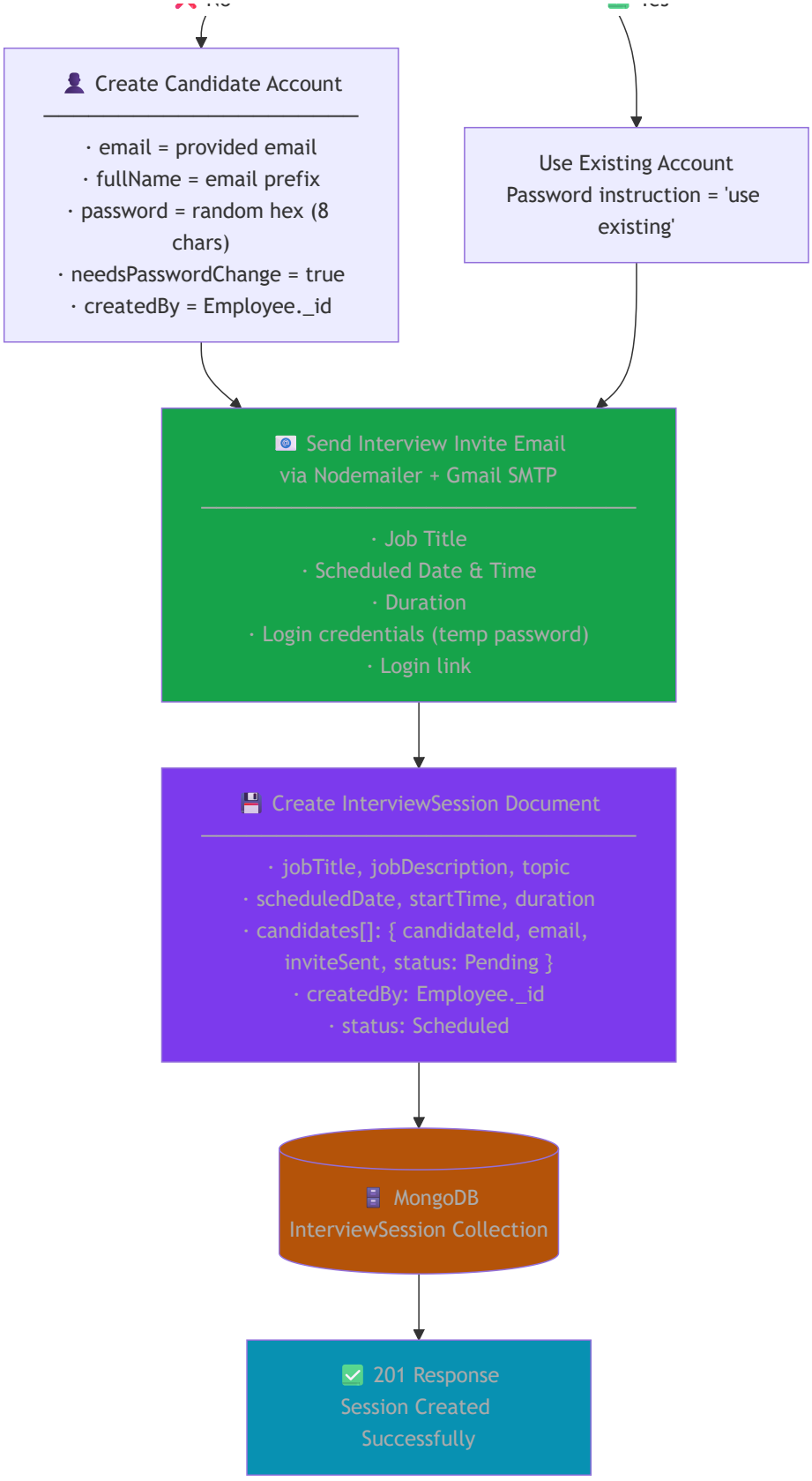
Layer	Technology	Purpose
File Storage	Cloudinary	Profile photo upload and hosting
Email	Nodemailer + Gmail SMTP	Interview invites, OTP, reschedule alerts

2. Phase 1 — Interview Session Creation

> **Who:** HR Employee | **When:** Before any candidate joins | **Outcome:** `InterviewSession` saved + emails dispatched

In this phase, the Employee fills out an interview creation form. Optionally, they can request the AI to generate a tailored system prompt based on the job description. For each candidate email provided, the system finds or creates an account, then dispatches a personalized invitation email containing temporary credentials.

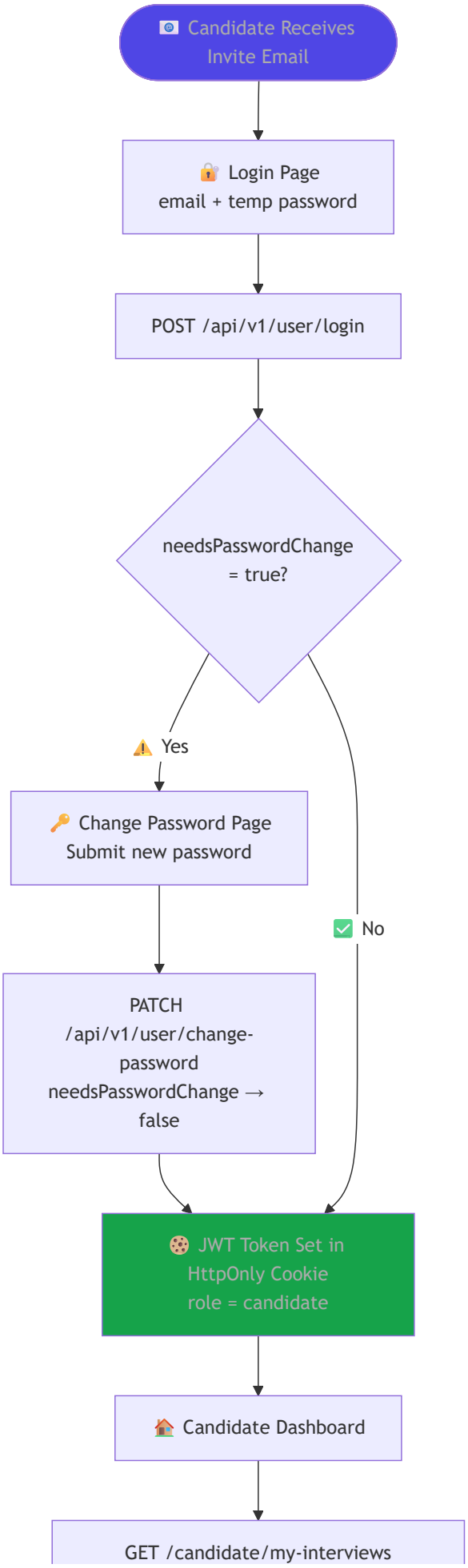

HR Employee Logs In
Interview Creation Form
· Job Title & Description
· Scheduled Date & Start Time
· Duration (minutes)
· Candidate Emails (comma-separated)
✨ Generate AI System Prompt?
Yes → POST /employee/interview/generate-prompt
No
Groq LLM Generates structured interviewer prompt based on Job Title + JD
POST /api/v1/employee/interview/create
For Each Candidate Email
User exists in MongoDB?
No
Yes

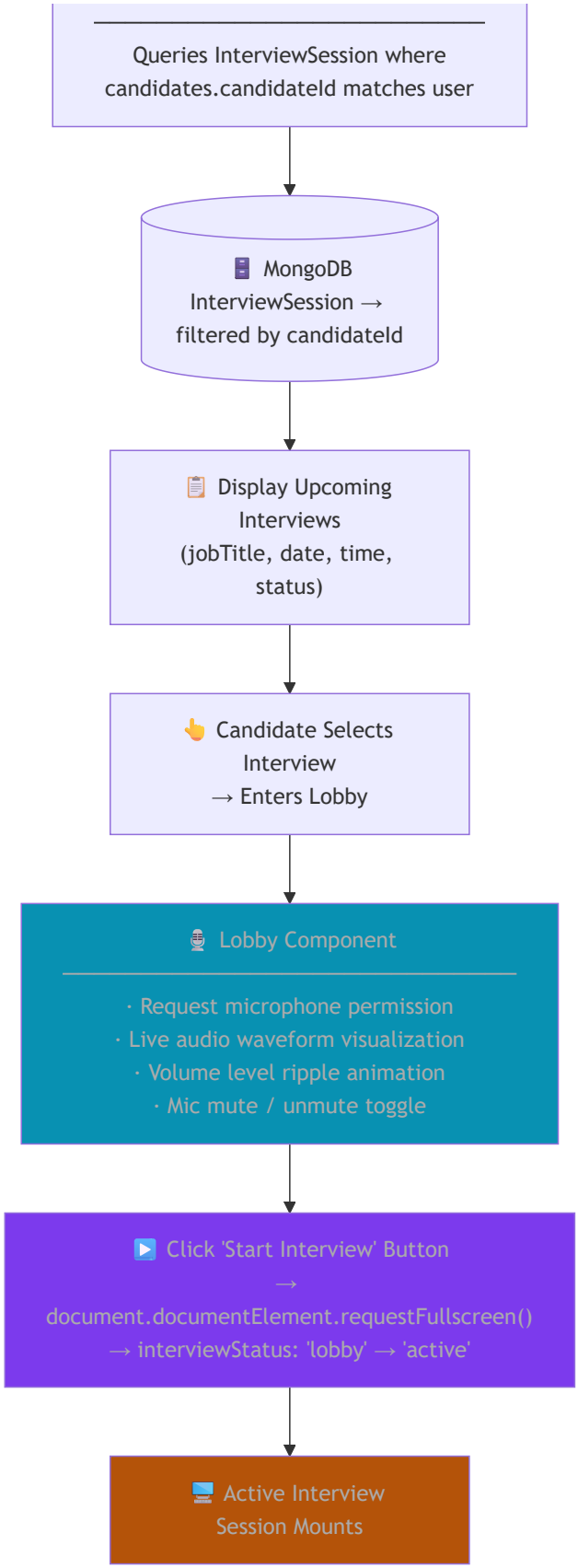


3. Phase 2 — Candidate Authentication & Lobby

> **Who:** Candidate | **When:** After receiving the invite email | **Outcome:** Candidate enters Lobby and performs Mic Check

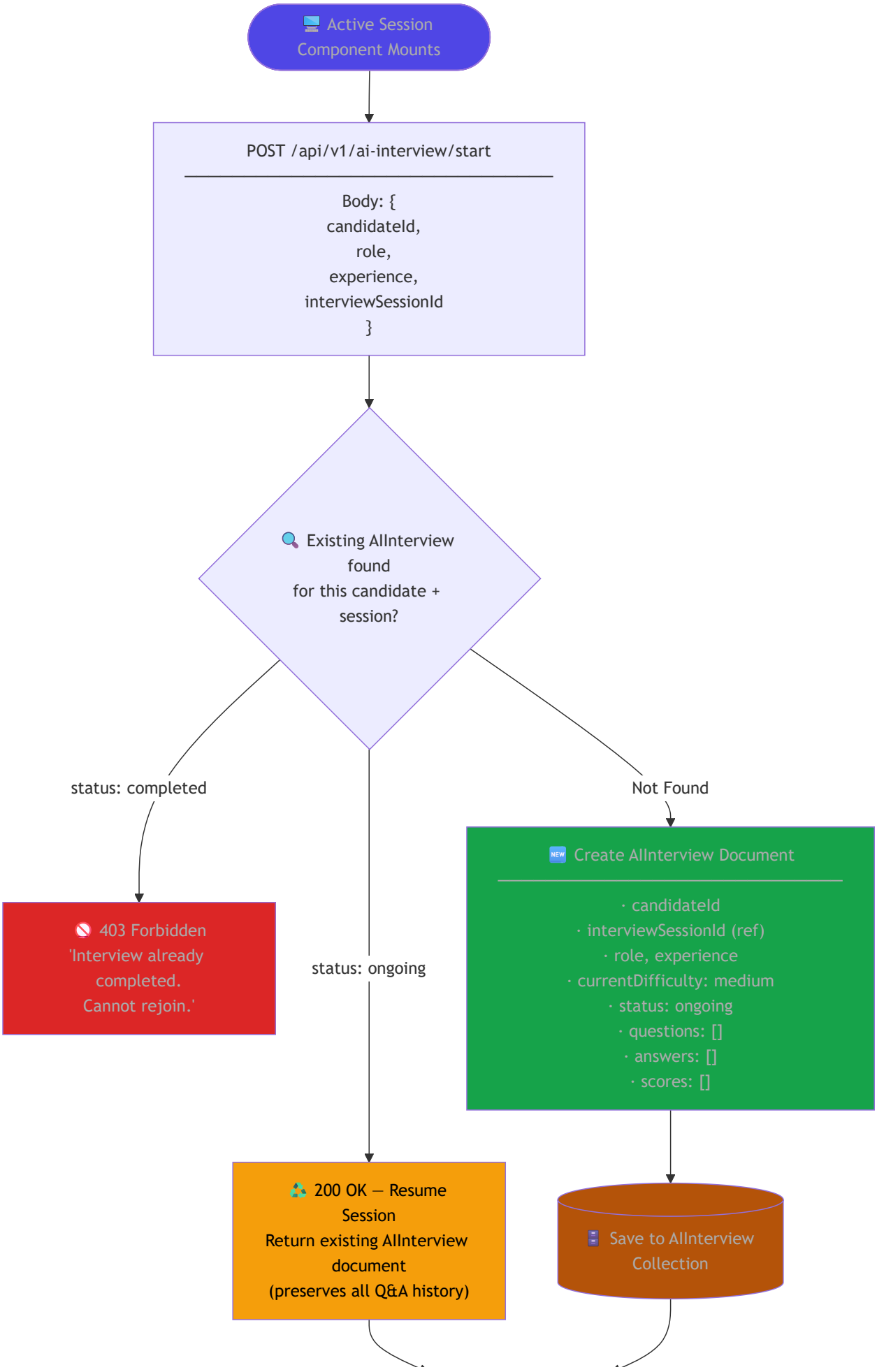
The candidate receives an email with credentials. On first login, they are forced to set a new password. Once authenticated, they see their scheduled interviews and enter a Lobby component that checks microphone health with a live audio visualizer before starting.

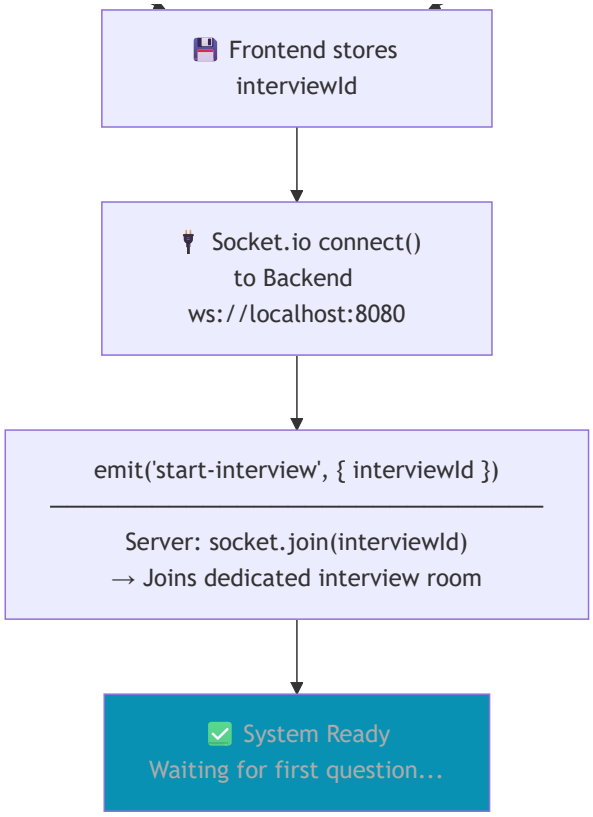

Candidate Receives Invite Email
Login Page email + temp password
POST /api/v1/user/login
needsPasswordChange = true?
Yes
Change Password Page Submit new password
No
PATCH /api/v1/user/change-password needsPasswordChange → false
JWT Token Set in HttpOnly Cookie role = candidate
Candidate Dashboard
GET /candidate/my-interviews



4. Phase 3 — AI Interview Initialization

> **Who:** Backend + Candidate Frontend | **When:** Active Session component mounts | **Outcome:** `AIInterview` document created, socket room joined, first question emitted

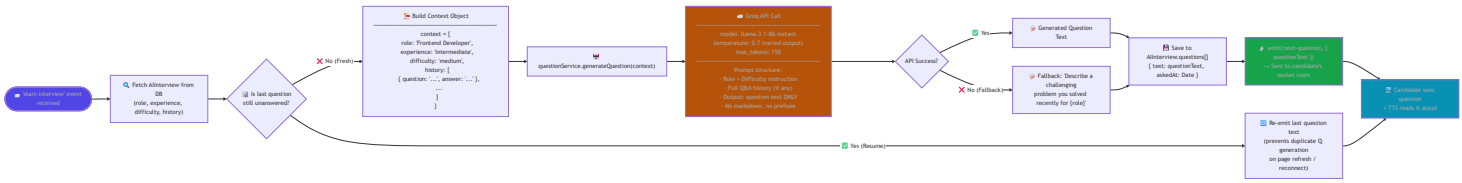

Active Session Component Mounts
POST /api/v1/ai-interview/start
Body: { candidateId, role, experience, interviewSessionId }
Existing AIInterview found for this candidate + session?
status: completed
403 Forbidden 'Interview already completed. Cannot rejoin.'
status: ongoing
Not Found
Create AIInterview Document
· candidateId
· interviewSessionId (ref)
· role, experience
· currentDifficulty: medium
· status: ongoing
· questions: []
· answers: []
· scores: []
200 OK — Resume Session Return existing AIInterview document (preserves all Q&A history)
Save to AIInterview Collection



5. Phase 4 — Real-Time AI Question Generation

> **Service:** `questionService.js` | **AI Model:** `llama-3.1-8b-instant` via Groq | **Temperature:** 0.7 (creative, varied)

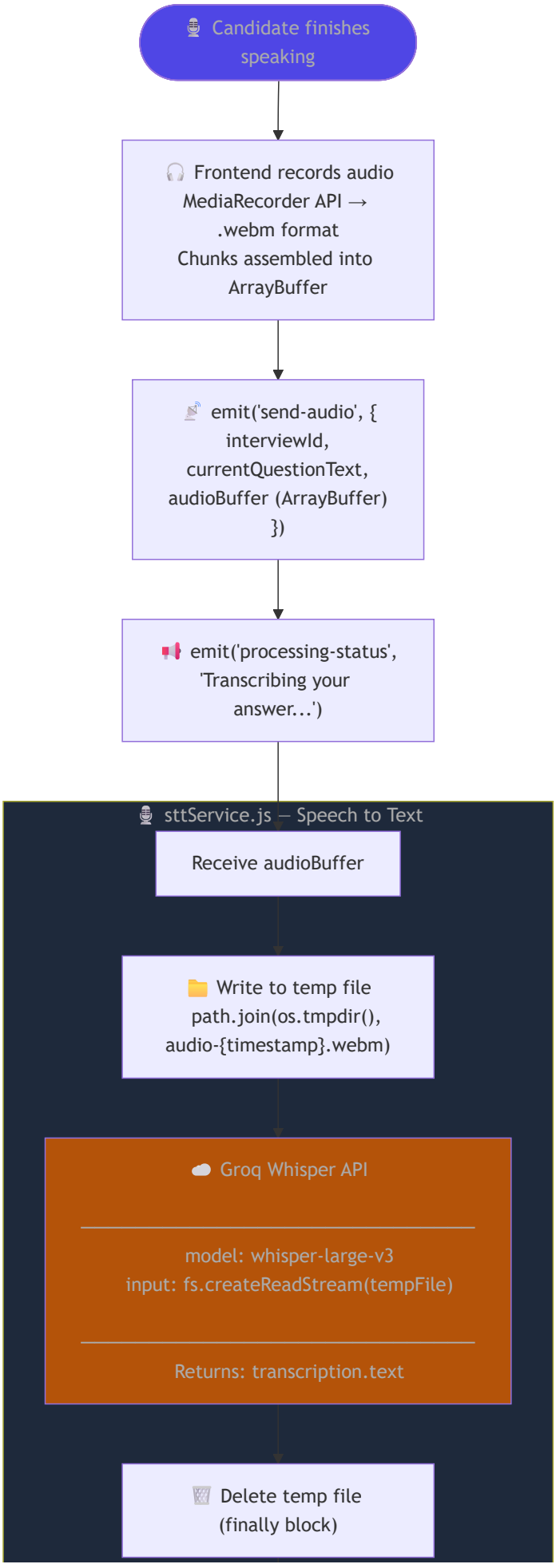
The question generation service is **context-aware**. It receives the full history of all previous questions and answers to ensure continuity, relevance, and non-repetition. The difficulty level is dynamically passed in and can change after every answer.

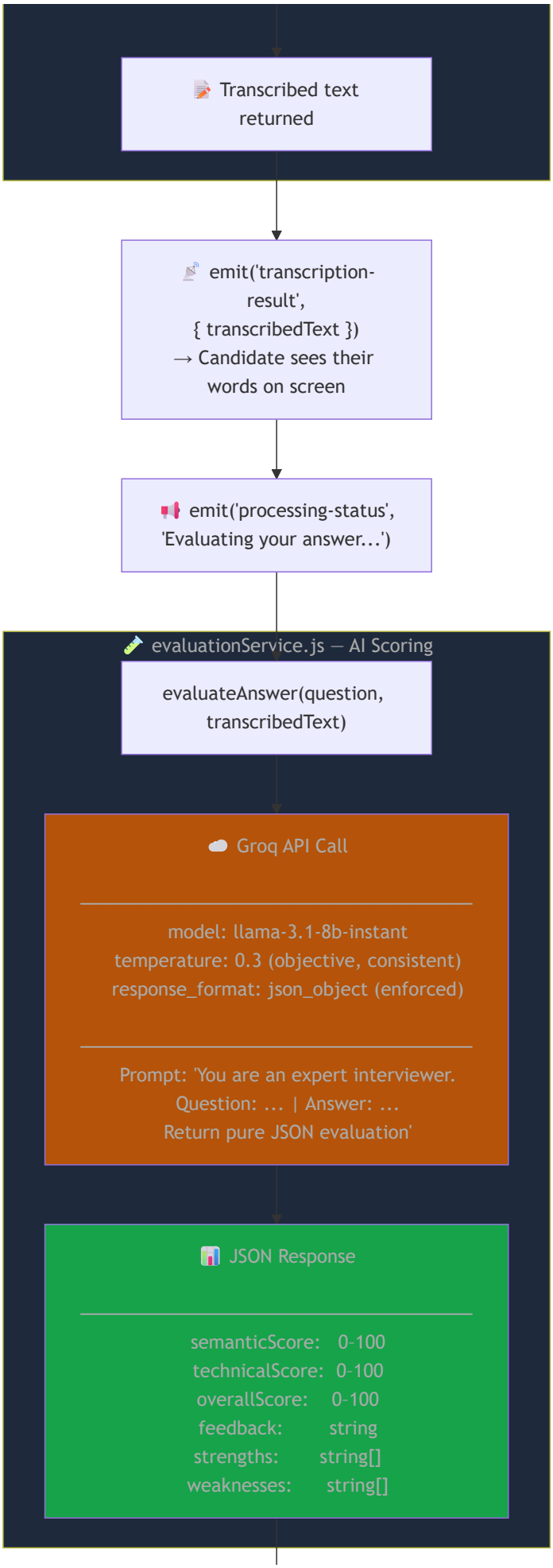


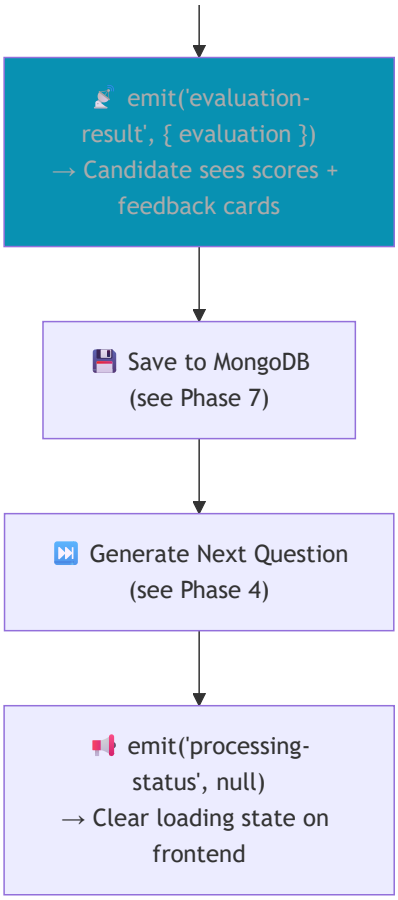
6. Phase 5 — Audio Answer Processing Pipeline

> **Services:** `sttService.js` + `evaluationService.js` | **Models:** Whisper large-v3 + llama-3.1-8b-instant | **Execution:** Sequential, real-time status emitted at each stage

This is the core AI pipeline. After the candidate finishes speaking, the raw `.webm` audio buffer is streamed through two AI services in sequence: first Speech-to-Text transcription, then AI-based evaluation against the question. All intermediate statuses are emitted in real-time to keep the UI updated.


Candidate finishes speaking
Frontend records audio
MediaRecorder API → .webm format
Chunks assembled into ArrayBuffer
emit('send-audio', {
interviewId,
currentQuestionText,
audioBuffer (ArrayBuffer)
})
emit('processing-status', 'Transcribing your answer...')
sttService.js — Speech to Text
Receive audioBuffer
Write to temp file
path.join(os.tmpdir(), audio-{timestamp}.webm)
Groq Whisper API
model: whisper-large-v3
input: fs.createReadStream(tempFile)
Returns: transcription.text
Delete temp file
(finally block)

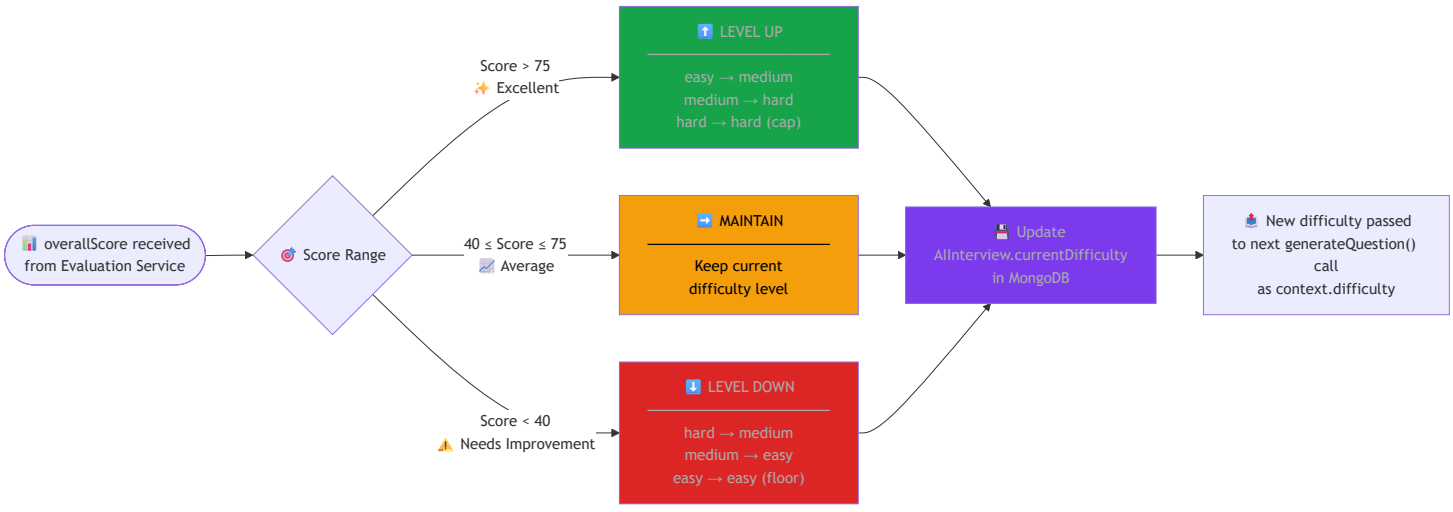

Transcribed text returned
emit('transcription-result', { transcribedText }) → Candidate sees their words on screen
emit('processing-status', 'Evaluating your answer...')
evaluationService.js — AI Scoring
evaluateAnswer(question, transcribedText)
Groq API Call
model: llama-3.1-8b-instant
temperature: 0.3 (objective, consistent)
response_format: json_object (enforced)
Prompt: 'You are an expert interviewer.
Question: ... | Answer: ...
Return pure JSON evaluation'
JSON Response
semanticScore: 0-100
technicalScore: 0-100
overallScore: 0-100
feedback: string
strengths: string[]
weaknesses: string[]



7. Phase 6 — Adaptive Difficulty Engine

> **Location:** `interview.socket.js` — `adjustDifficulty()` function | **Trigger:** After every answer is evaluated

The system adapts difficulty in real-time based on performance. This prevents interviews from being too easy for strong candidates and too discouraging for weaker ones — ensuring a fair, dynamic assessment at all levels.



Difficulty Level Scale

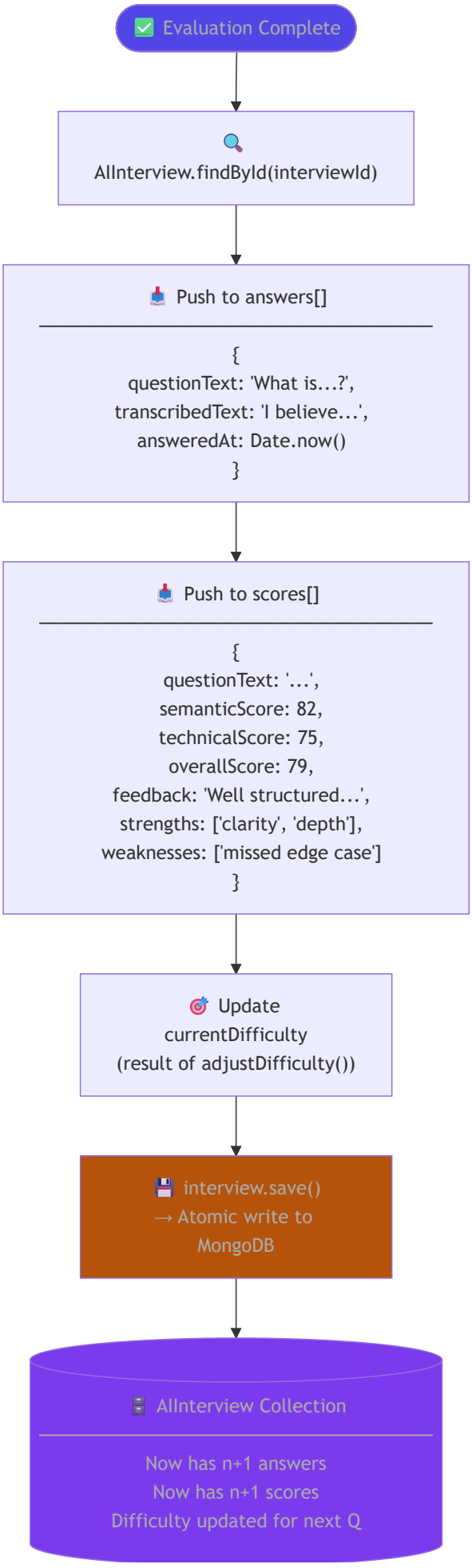
Score	Label	Effect	Behavior
> 75	Excellent	Level UP	Next question at higher difficulty

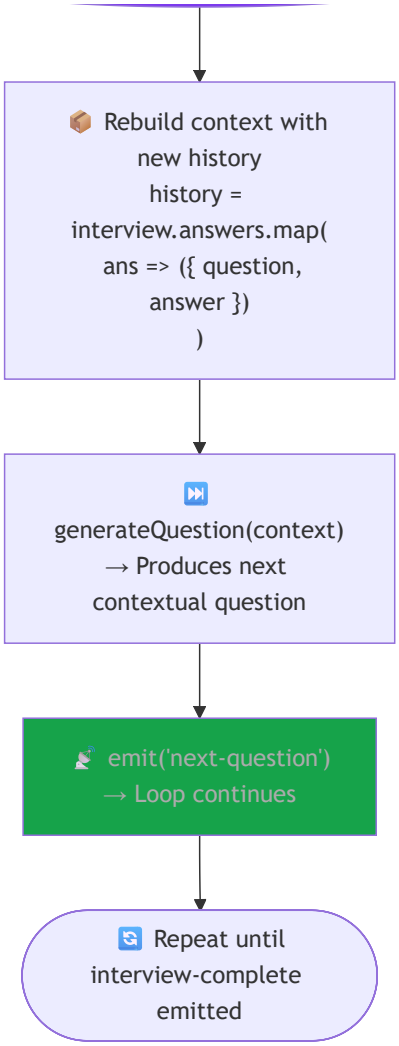
Score	Label	Effect	Behavior
40–75	🟡 Average	Maintain	Same difficulty maintained
< 40	🔴 Needs Work	Level DOWN	Next question at lower difficulty

8. Phase 7 — Database Write After Each Answer

> **Trigger:** Successfully processed audio | **Target:** `AIInterview` MongoDB document

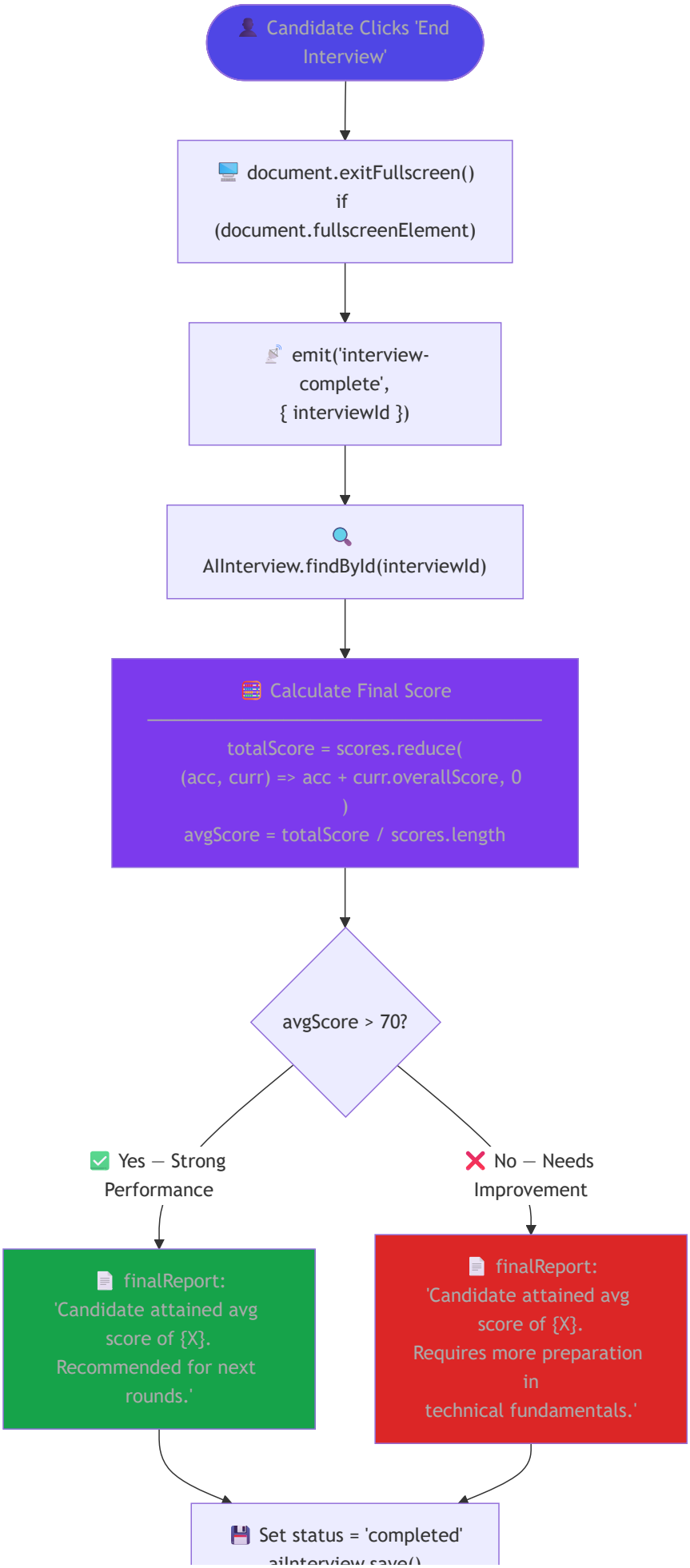
After every evaluation cycle, three sub-arrays in the `AIInterview` document are updated atomically: `answers`, `scores`, and `currentDifficulty`. The document acts as the single source of truth for the entire interview session.

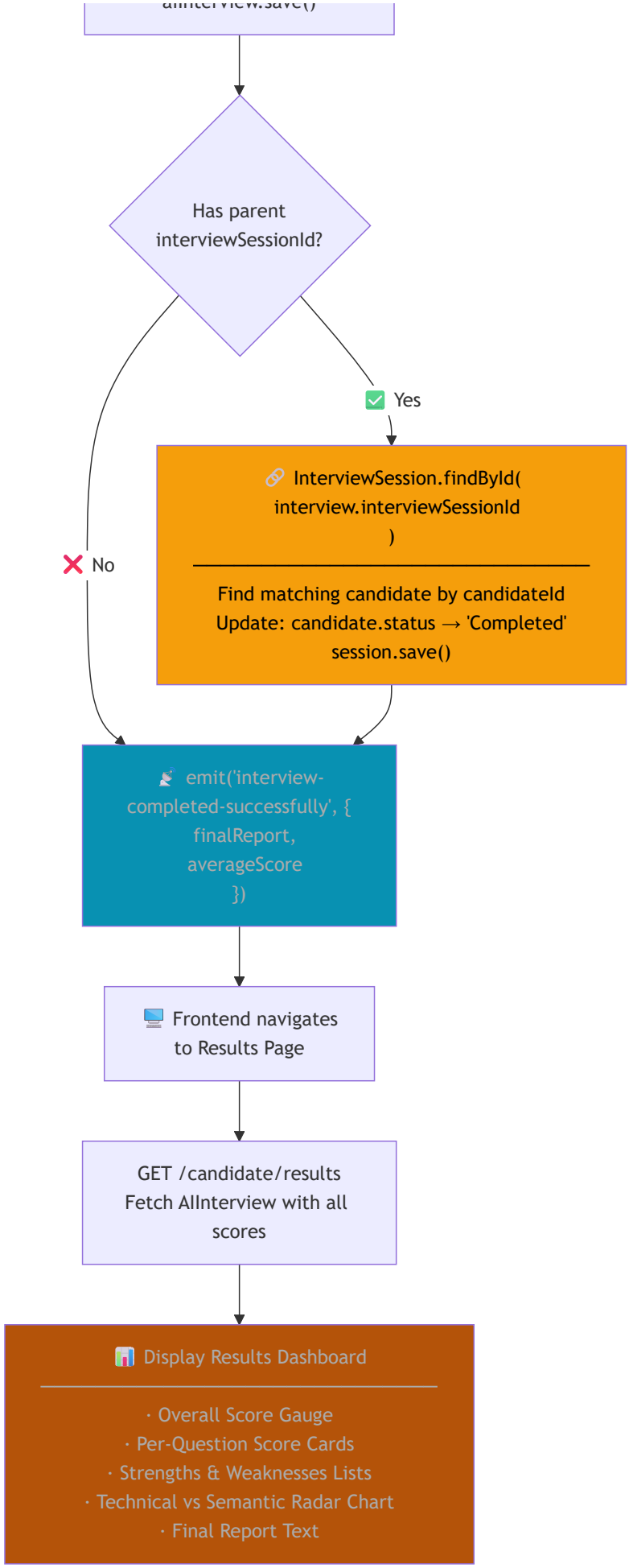

✅ Evaluation Complete
🔍
AllInterview.findById(interviewId)
📥 Push to answers[]
{
questionText: 'What is...?',
transcribedText: 'I believe...',
answeredAt: Date.now()
}
📥 Push to scores[]
{
questionText: '...',
semanticScore: 82,
technicalScore: 75,
overallScore: 79,
feedback: 'Well structured...',
strengths: ['clarity', 'depth'],
weaknesses: ['missed edge case']
}
🎯 Update
currentDifficulty
(result of adjustDifficulty())
💾 interview.save()
→ Atomic write to
MongoDB
🗄️ AllInterview Collection
Now has n+1 answers
Now has n+1 scores
Difficulty updated for next Q



9. Phase 8 — Interview Completion & Final Report

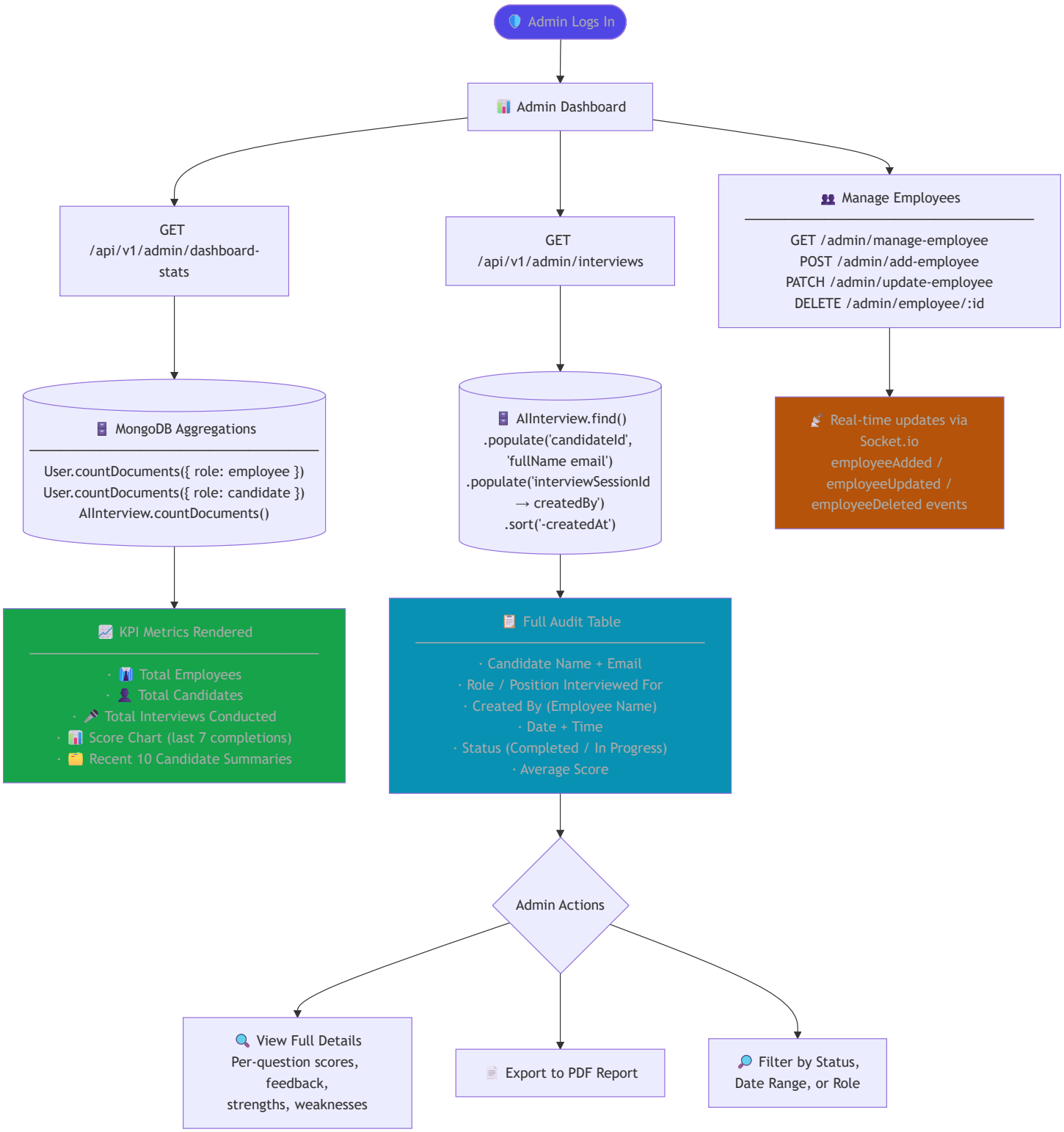
> **Trigger:** Candidate emits `interview-complete` | **Outcome:** Final score computed, `InterviewSession` status synced, results rendered


Candidate Clicks 'End Interview'
document.exitFullscreen() if (document.fullscreenElement)
emit('interview-complete', { interviewId })
AIInterview.findById(interviewId)
Calculate Final Score
totalScore = scores.reduce((acc, curr) => acc + curr.overallScore, 0)
avgScore = totalScore / scores.length
avgScore > 70?
Yes — Strong Performance
No — Needs Improvement
finalReport: 'Candidate attained avg score of {X}. Recommended for next rounds.'
finalReport: 'Candidate attained avg score of {X}. Requires more preparation in technical fundamentals.'
Set status = 'completed'

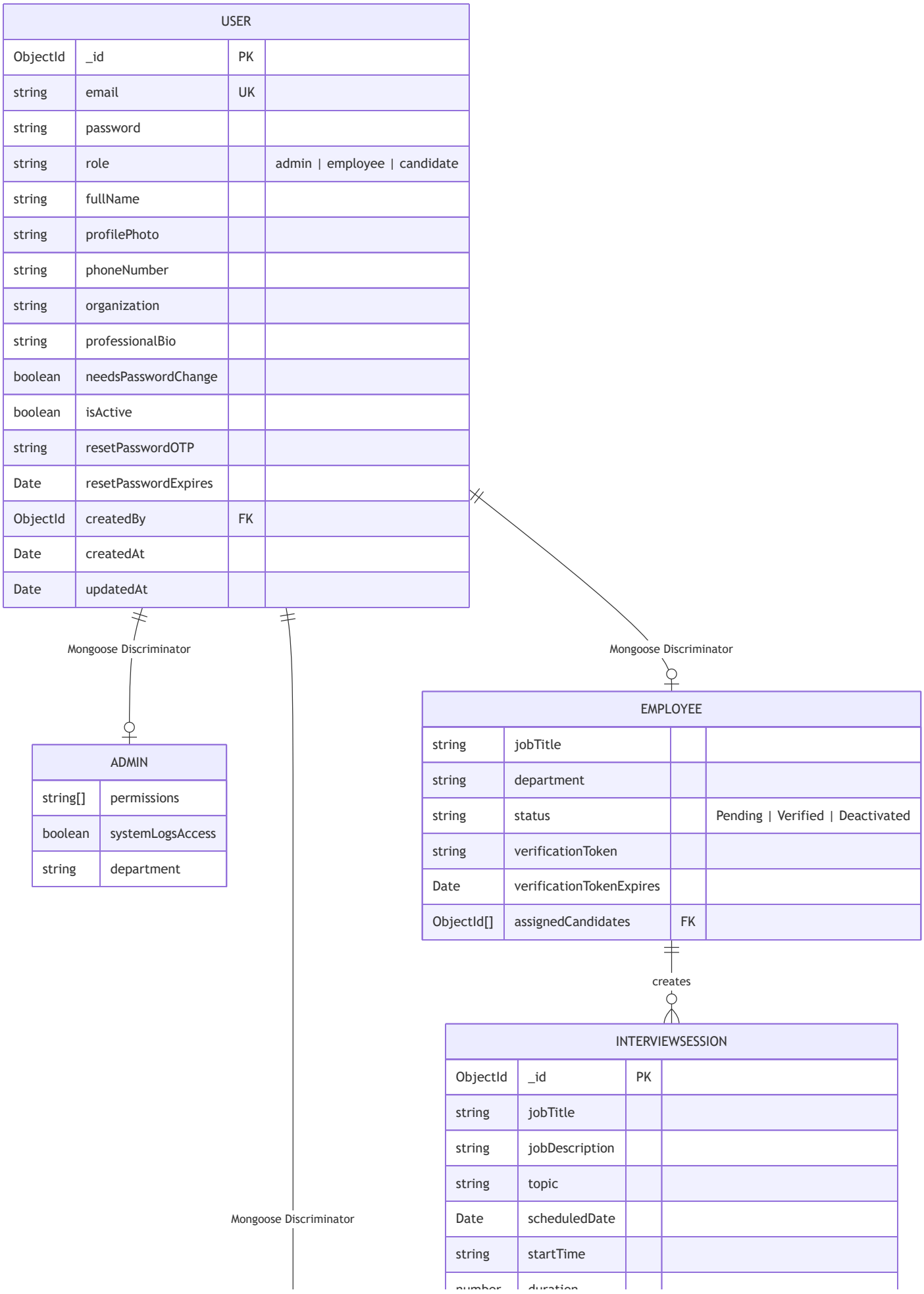

Has parent interviewSessionId?
✅ Yes
❌ No
🔗 InterviewSession.findById(interview.interviewSessionId)
Find matching candidate by candidateId
Update: candidate.status → 'Completed'
session.save()
🎉 emit('interview-completed-successfully', { finalReport, averageScore })
🖥️ Frontend navigates to Results Page
GET /candidate/results
Fetch AIInterview with all scores
📊 Display Results Dashboard
· Overall Score Gauge
· Per-Question Score Cards
· Strengths & Weaknesses Lists
· Technical vs Semantic Radar Chart
· Final Report Text

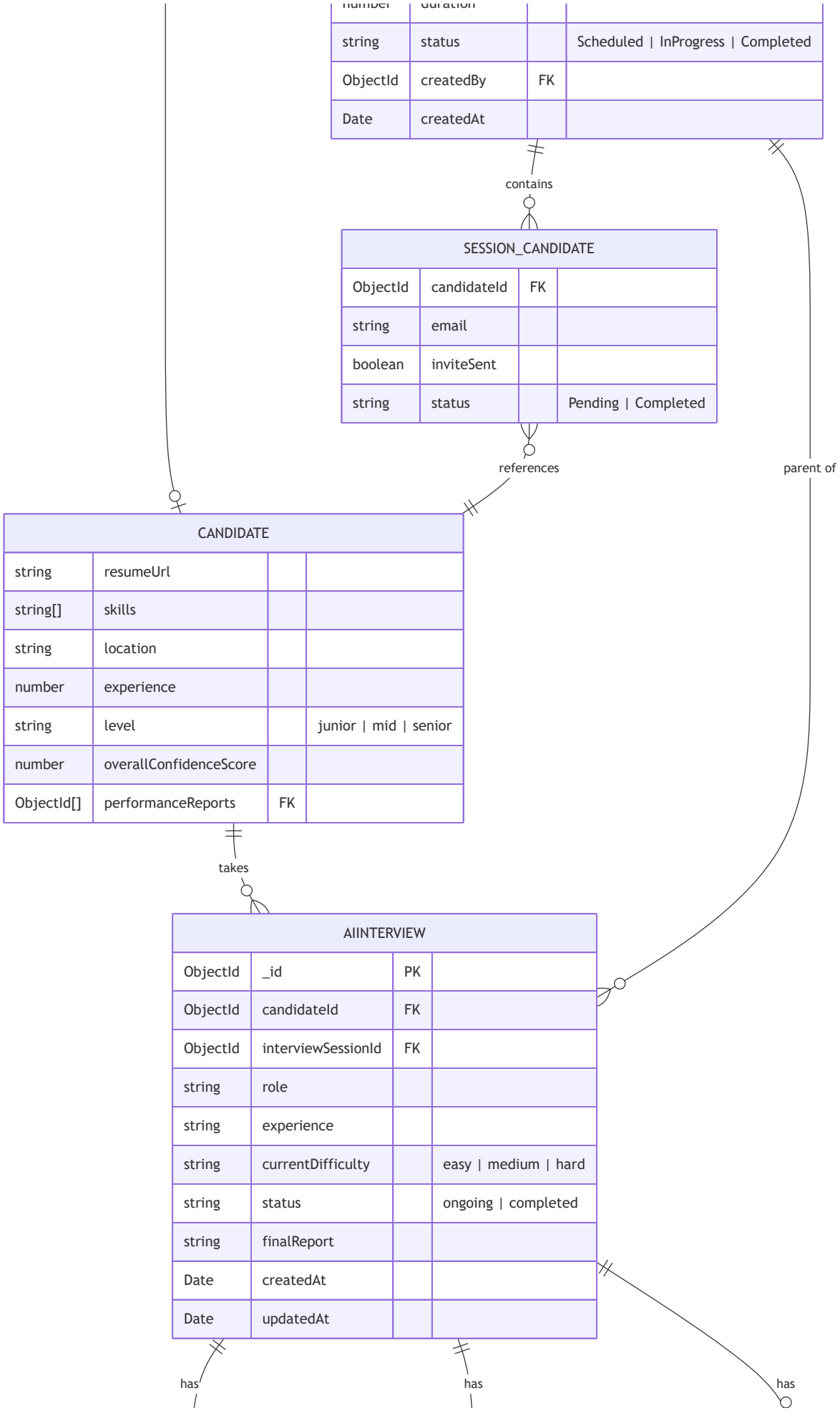
10. Phase 9 — Admin Audit & Monitoring

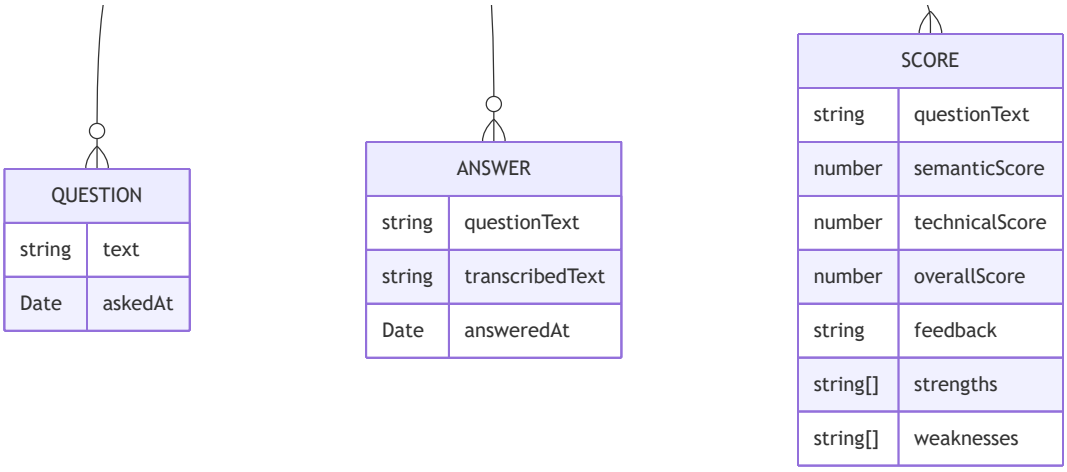
> **Who:** Admin | **When:** Any time after interviews are conducted | **Purpose:** System-wide reporting and employee management



11. Complete Data Model Relationships


USER
ObjectId _id PK
string email UK
string password
string role admin | employee | candidate
string fullName
string profilePhoto
string phoneNumber
string organization
string professionalBio
boolean needsPasswordChange
boolean isActive
string resetPasswordOTP
Date resetPasswordExpires
ObjectId createdBy FK
Date createdAt
Date updatedAt
Mongoose Discriminator
Mongoose Discriminator
Mongoose Discriminator
ADMIN
string[] permissions
boolean systemLogsAccess
string department
EMPLOYEE
string jobTitle
string department
string status Pending | Verified | Deactivated
string verificationToken
Date verificationTokenExpires
ObjectId[] assignedCandidates FK
creates
INTERVIEWSESSION
ObjectId _id PK
string jobTitle
string jobDescription
string topic
Date scheduledDate
string startTime


number
duration
string
status
Scheduled | InProgress | Completed
ObjectId
createdBy
FK
Date
createdAt
contains
SESSION_CANDIDATE
ObjectId
candidateId
FK
string
email
boolean
inviteSent
string
status
Pending | Completed
references
parent of
CANDIDATE
string
resumeUrl
string[]
skills
string
location
number
experience
string
level
junior | mid | senior
number
overallConfidenceScore
ObjectId[]
performanceReports
FK
takes
AIINTERVIEW
ObjectId
_id
PK
ObjectId
candidateId
FK
ObjectId
interviewSessionId
FK
string
role
string
experience
string
currentDifficulty
easy | medium | hard
string
status
ongoing | completed
string
finalReport
Date
createdAt
Date
updatedAt
has
has
has



12. Complete Socket.io Event Reference

📤 Client → Server Events

Event Name	Payload Schema	Description	Phase
`start-interview`	`{ interviewId: string }`	Join the interview room and receive the first question	Phase 3
`send-audio`	`{ interviewId: string, currentQuestionText: string, audioBuffer: ArrayBuffer }`	Submit candidate's voice answer for transcription and evaluation	Phase 5
`interview-complete`	`{ interviewId: string }`	Signal the end of the interview and trigger final report generation	Phase 8

📥 Server → Client Events

Event Name	Payload Schema	Description	Phase
`next-question`	`{ questionText: string }`	AI-generated question, delivered after start or each answer	Phase 4
`transcription-result`	`{ transcribedText: string }`	Real-time speech-to-text result shown on screen	Phase 5

Event Name	Payload Schema	Description	Phase
`evaluation-result`	`{ evaluation: ScoreObject }`	Full AI evaluation with scores, feedback, strengths, weaknesses	Phase 5
`processing-status`	`{ message: string | null }`	Loading state message during transcription/evaluation, null clears it	Phase 5
`interview-completed-successfully`	`{ finalReport: string, averageScore: number }`	Signals the end of interview with final summary	Phase 8
`interview-error`	`{ message: string }`	Error emitted for any pipeline failure	All
`employeeAdded`	`Employee`	Broadcasts new employee document to admin dashboard	Admin
`employeeUpdated`	`Employee`	Broadcasts updated employee document	Admin
`employeeDeleted`	`string (id)`	Broadcasts deleted employee ID	Admin

ScoreObject Schema

```
{
  "semanticScore":   75,
  "technicalScore":  80,
  "overallScore":    77,
  "feedback":        "Good conceptual understanding with clear communication...",
  "strengths":       ["Clear explanation", "Relevant example provided"],
  "weaknesses":      ["Missed time complexity discussion"]
}
```

📌 REST API Quick Reference

Method	Endpoint	Auth	Role	Purpose
`POST`	`/api/v1/user/login`	❌	All	Login and receive JWT cookie
`POST`	`/api/v1/user/register`	❌	Admin	Register new admin account
`PATCH`	`/api/v1/user/change-password`	✅	All	Update first-time password
`POST`	`/api/v1/employee/interview/create`	✅	Employee	Create interview session
`GET`	`/api/v1/employee/interviews`	✅	Employee	List own interview sessions
`GET`	`/api/v1/employee/interview/:id`	✅	Employee	Get session details
`PATCH`	`/api/v1/employee/interview/:id`	✅	Employee	Update / reschedule session
`POST`	`/api/v1/employee/interview/:id/candidate`	✅	Employee	Add candidate to session

Method	Endpoint	Auth	Role	Purpose
DELETE	/api/v1/employee/interview/:id	✅	Employee	Delete session
POST	/api/v1/employee/interview/generate-prompt	✅	Employee	Generate AI system prompt
GET	/api/v1/candidate/my-interviews	✅	Candidate	Fetch assigned interviews
POST	/api/v1/ai-interview/start	✅	Candidate	Initialize AI interview session
POST	/api/v1/ai-interview/end	✅	Candidate	Mark session as completed via REST
GET	/api/v1/admin/dashboard-stats	✅	Admin	System-wide KPI metrics
GET	/api/v1/admin/interviews	✅	Admin	All AI interviews (audit)
POST	/api/v1/admin/add-employee	✅	Admin	Invite new employee
PATCH	/api/v1/admin/update-employee	✅	Admin	Update employee record
DELETE	/api/v1/admin/employee/:id	✅	Admin	Remove employee

VIRQA AI Interview Platform

Final Year Project — University of Central Punjab (UCP) — F22

Built with ❤️ by Muhammad Ummar

Document generated from live source code analysis — all flows reflect actual implementation